

Lec 15 Regularization Techniques in AE: Contractive

Lec 15 Regularization Techniques in AE: Contractive

Lecture	15 / 63 + intro
Duration	27:05
Captions	English (auto-generated)
Words	3,751
Video	87pbybKetu4
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome to the lecture series on fundamentals of generative AI and the large language models. In the previous two sessions, we had seen a regularization techniques such as denoising and then the sparse. In this particular session, we will see the third technique called contractive autoencoders. So, what exactly is a contractive autoencoders? Before that, let us see what exactly is autoencoders. So, the overall working mechanism of autoencoder is when an input is a passed to an encoder, encoder is going to give us the latent representation, which is an hidden representation. From this a hidden representation, we want the we want the reconstruct we want to reconstruct the input well.

[00:48] So, the autoencoder says from the hidden representation, reconstruct the input well. But, what is a contractive autoencoders says that reconstruct the input well, but to keep the hidden representation a stable. Now, what exactly mean by stable here is suppose we have two inputs. Let us say take those two inputs are these two. Now, I'll pause here for few seconds. Observe to it very carefully. What is the difference between these two images?

[01:20] So, there is absolutely no difference with respect to features, right? The difference is only because there is some noise that is being added to an other image. So,

when the input remains similar, when two input samples differ very slightly, their hidden representation should remain very close or their hidden representation should remain very similar. That is our requirement. So, the contractive autoencoders mainly focuses on of course, reconstruction has to be there, but keep the hidden representation stable. The meaning of stable here is when two input samples differ slightly, their hidden representation should remain similar.

[02:04] Now, let us take this is the original input X . To this original input, we have added a small change. That a small change is denoted with this particular return. For the original input, there can be a small noise that has been added or it can be an uh brightness. There is a change in the brightness. So, if at all, if you have an input which is a slightly varying, then their hidden representation should not differ much. But, when you give this kind of an input to an auto encoders, we can definitely see some change in the hidden representation. What does it mean? Suppose when X is a passed to an auto encoder, it is going to give the representation, right? Now, when this noisy data, when it is given to an auto

[03:04] encoder, it is going to give the hidden representation. So, what is that hidden representation is capturing all the you know meaningful features in the from the input. That compact representation is what is the hidden representation or the latent representation. When we look into an auto encoders, we will definitely auto encoders produce There there will be some difference between the hidden representation when you pass these two as an input. But, the difference is only with respect to a small change. That change is either due to noise or it can be due to change in the brightness. In that case, what is that contractive auto encoders says?

[03:51] The hidden representation of the input and the hidden representation of a small changed input should remain nearly the same. That is what is contractive autoencoders ensures. Therefore, the main goal here is of course you should get a good reconstruction, but at the same time we should have a stable hidden representation. So, this is what is the overall theoretical concept. The concept here is when there is a slight change in the input, when you look into this, there is a slight change in the input, then the hidden representation should not vary much is what is all what contractive autoencoders ensures. If that is the case, how do it achieves it?

[04:41] So, mainly it is a responsible to prevent an overcomplete autoencoder. It is mainly designed so that, you know, this trivial identity mapping should be avoided in an overcomplete autoencoder. So, it is an regularized autoencoder designed to prevent an overcomplete autoencoder from learning the trivial identity mapping. How is that we are achieving? We are achieving by adding a Jacobian based a regularization term to the loss function. So, what [snorts] exactly is a Jacobian based? How is that we are denoting the Jacobian based? The Jacobian based regularization term is represented with this particular you know, math. This is the mathematical formula for the Jacobian based regularization term. Now, in this what

[05:34] exactly each of these terms represents? » [snorts] » And thus, this J of X given for H , you know, J this particular um is this a denotes the Jacobian of hidden representation H with respect to input X . So, this is what this particular term denotes. It is the Jacobian of all the hidden units H with respect to input X . In this, H is denoted as G of X . Now, can you imagine what exactly is this H ? Now, hidden representation if you take, then you first have the summation, you will take the weighted sum of input and then the bias.

[06:25] Then you will be passing it to an activation function. That is what is this G of X . That is what is the hidden representation. The output is H . H is whatever the summation that we are going to get, that is passed to an activation function, that is G of X . Where H is the hidden representation, X is the input, and this term contains the partial derivatives of all the hidden units with respect to all the input dimension. Now, if you have to understand that this with one diagram, let us let me give one example to you. We have the input and then we have an hidden representation.

[07:12] Let us consider this as X_1, X_2, X_3 . What are these? These are all three features in the data set. Right? And this is H_1, H_2, H_3 , and H_4 . So, what is this? It is the partial derivatives of all these hidden units with respect to all of these input dimension. So, how is that we denote? We denote it um like this. It is a matrix. What the matrix? It is a partial derivative of all these hidden units with respect to all of the input dimension.

[07:51] Now, um in this, the first particular column, what is this first column says? Now, if you are representing if you have to establish a connection, is it not you will establish a connection like this? And this is a past here also. This is past. Is it not we establish a connection in this way? So, here we are if we are taking this derivative, the first column, what exactly it says? How much the hidden unit H_1 is changing when there is a small change in X_1 . How much hidden unit H_2 changes when there is a small change in X_1 . How much hidden unit H_3 is changing when there is a change in X_1 . And the similar way with the H_4 .

[08:41] So, the first column says the partial derivatives of all of these hidden units with respect to the input feature X_1 . And what are we measuring? We are measuring how sensitive each of these hidden units change when there is a small change in X_1 . That is the input feature X_1 . So, this is what the Jacobian matrix says. Now, what exactly is this matrix saying? Now, just imagine there is an input. Now, you have made a small change in the input. For example, let's take one more example.

[09:22] Since there are three features, one data is 1.5, 2.5, 3.5. Right? Then you have an you have made a small change in this. So, instead of 1.5, you have made 1.6, 2.5, 3.4. There is a small change in this input. When there is a small change in the input, hidden representation should not vary too much. How do you ensure how are we measuring that hidden unit is a changing a much, hidden unit is not changing

much? Is it not this particular matrix is telling us? Each of this element in this matrix says how much a hidden unit H1 is changing when there is a change in X1. How much hidden unit H1 changes when there is a change in X2. How much hidden unit H1 changes when there is a change in X3. So, all of these are being

[10:21] captured in this particular matrix, and that matrix we call it as an Jacobian matrix. Now, in this particular Jacobian based regularization term, we have this concept also. What is this? It is sum of square of all the elements which are represent in this particular Jacobian matrix. So, we call it as an Frobenius norm. So, it is the sum of the square of all the elements which are present in the Jacobian matrix. Now, if this particular value is very high, what is the meaning of it?

[11:02] Which means each of these elements in this matrix, the values are high. If [snorts] the values are high, the square also will be more. So, what is the meaning here? Every of the hidden unit is changing too much for a small change in the input, right? So, that is what a large Jacobian value means. Hidden representation changes a lot even for a small change in the input. But, we don't want that. Contractive autoencoder always tries to make these Jacobian values so small so that hidden representation remain more stable.

[11:42] So, I hope the intuition is clear. So, let us try to get a little more in detail. This is called an Jacobian based regularization term. This regularization term is being added to the reconstruction loss. So, what we are doing here? The term is added to the original loss function. And what is our overall aim? The overall aim is the loss should be minimized. If the loss has to be minimized, definitely this second term value should be small. If the second term value is a small, what is the meaning of it? That hidden representation should not change it too much for a small change in the input.

[12:30] Therefore, the second term will be ideally close to zero. Now, let's take one term. So, what is one term here? The that is derivative of H1 with respect to X1. So, which means if you have to go back to our diagram, we had the feature X1, X2, and X3. We had an hidden representation H1. So, you have connected to this. Now, we are seeing how much H1 gets changed when there is a small change in the input X1, the feature X1. Now, what are the possibilities that this particular term can take? Either the value is high or the value is low, right? If the value is very high, what is the meaning of it? Can you think?

[13:24] Derivative of H1 with respect to X1, the value is high. What is the meaning? The meaning is very simple, which means H1 is changing a lot for a small change in X1, which means H1 is very much sensitive to that X1. That sensitivity here meaning that X1 even for a small change in X1, H1 is reacting to it. So, here the value when derivative of this H1 with respect to X1, if this value is a large, it means for a small change in X1 causes a change in H1.

[14:07] Right? The other meaning is it is very sensitive to X1. So, hidden neuron

is reacting strongly for a small input change. We don't want this, right? Because for a small change in the input, if hidden representation changes, then ultimately reconstruction also captures that noise, right? So, we don't want that. So, at the other side, what if the value is very small? That is this value is a very small value, which means that even for a small change in X_1 , H_1 is not reacting much, which means H_1 is completely insensitive to the variations in X_1 .

[14:53] So, insensitive here meaning that hidden neuron reacts very little to a small change in the input. Ultimately, contractive autoencoder mainly does not want H_1 to change too much for unimportant changes in X_1 . So, this is the overall meaning. This particular slide is a very important where we are finally understanding how exactly it is achieved. Now, you see look into this expression. What is the overall intuition behind?

[15:33] You have an input. Input is a pass through an encoder and encoder has to capture the meaningful representation and that meaningful representation is the hidden representation. Now, that hidden representation is a pass as an input to to an decoder. Now, what is the role of a decoder? The role of a decoder is is to reconstruct. Now, what is the overall contract to autoencoder is a saying? Don't capture the information if there is a small change in the input. Now, just to see if a hidden representation does not capture the information, then can you can we say hidden representation is poor? If hidden representation is a poor, what will be reconstruction one?

[16:26] Definitely it will be poor. When that reconstruction when you compare it with the original input, is it not the loss is too much? So, here the first step term, this particular term says keep enough information so that the model can reconstruct the input well. Do you agree to this? Reconstruction is possible when the hidden representation is good. Right? So, the first term says keep as much information as possible so that the model can reconstruct. What is the second term is a saying?

[16:59] What is the second term here? The hidden representation should not change much for a small variation in the input. Now, how will the hidden representation knows that you know it is a very small change, I should not react too So, but second term is a saying do not let the hidden representation change it too much. If that is the case, is it not two contradictory objectives? One is saying keep as much information as possible so that the reconstruction is good.

[17:33] The second term is a saying don't keep so much information. Don't react for every small change. If you doesn't react, then how reconstruction is possible? So, is it not two contradictory? So, the trade-off here is preserve only the most important variations in the data while less important variations has to be ignored or suppressed. That is the trade-off here. So, one is saying keep more important, one is saying don't keep. So, what is the compromise here? What is the trade-off here? Trade-off is

preserve only the most important variations while suppressing the unimportant variations.

[18:18] So, therefore, what exactly is the model problem? It cannot keep every detailed information. At the same time, it cannot be completely insensitive at the same time. Right? Therefore, here the hidden representation should learn the balance to one. How that balance is achieved? By using a regularization parameter λ . So, here the λ will decide how strongly we should punish the hidden representation if it is changing too much. So, here the value can be lower value or the λ will be a higher value. If it is a higher value, we achieve more stable features. If it is a lower value, we get a detailed reconstruction. Okay? So, now for the next 2-3 minutes, whatever I'll be explaining, this is very important to

[19:11] you. So, now I'll ask you certain questions, you can answer to yourself. And with this, you'll be able to understand this concept more clearly. Now, let us assume two scenarios I'll give you. The first scenario is there is a some variation which has occurred in the input. Now, those variations are very important variations. What do you mean by important variations or genuine variations which is a part of original data? It is not because of some noise, it is not because of some change in the brightness. No, that variation is a genuine variation. Which means what is the ideal requirement? Our hidden representation has to capture that.

[19:58] Now, what if hidden representation does not capture? Because we have been trained that hidden representation should not react much. So, now there is an important genuine change in the input. Variation in the input. Hidden representation ideally has to react to it. That means it has to capture. What if it doesn't capture? What is going to happen? Can you answer to yourself? What if it doesn't capture? It is going to give a poor reconstruction.

[20:31] How do you understand that it is a poor reconstruction? At the end, you are comparing it with the original. Reconstructed one, you are comparing it with the original. So, when you compare it with the original, that important variation what you have seen in the input is not there here because model did not capture. Why model did not capture? Because we have trained such that hidden representation should not react much.

[20:55] So, in this case, is it not the reconstruction loss has increased? And then the penalty term has drastically reduced because hidden representation is not at all changing. When hidden representation is not changing, all the derivative terms will be low. When derivative terms will be low, then your reconstruction value has increased. So, from this a scenario, what is that the model is understanding? So, it is an important variation. I should capture that. Hidden representation has to capture. Is it not this information we are conveying? I'm summarizing this one point. Whenever there are a genuine and valid variations occurs in the input, hidden repre-

sentation should react to it. What do you mean by react? Hidden

[21:42] representation has to capture. How is that hidden representation understands that I have to capture it? Because if it doesn't capture and if it goes for reconstruction and the reconstructed output when you compare it with the original input, definitely those important variations wouldn't have been occurred in the reconstruction one. In that case, the reconstruction loss will be very high. When reconstruction loss is very high, it is the meaning of that the variations are important, capture it.

[22:12] I hope this analogy is clear. That is what is being explained here. Now, let us take the second scenario. What is the second scenario? Now, let us assume there is an image and that image there is a small variation. Let us take this example. Look at this example here. There is a small variation in the input, which means there is a noise which has occurred in the image. But, original features remained the same. Can you look into both the images? Only the small noise has been added. Ideally, what we have to do this? Hidden representation should not capture, correct? So, now let us consider this scenario.

[22:58] You have passed this input. Now, since there is a small variation, ideally, we want the hidden representation not to react, right? Now, let us assume hidden representation has reacted. What do you mean by reacted? For every small change in the input, hidden representation has captured that information, which means our derivative of $H1$ with respect to $X1$ has increased. So, if it has increased, what VAE tells, it should not, correct?

[23:27] Okay. Now, it has increased. Now, which means it has captured every detailed information. Now, that when you pass it to a decoder and if you allow the decoder to reconstruct, is it not that noise is also been reconstructed? Right? It is also been added in the reconstructed one. Now, that when you compare it with the original input, is it not the loss is more? Agree? The loss is more. And at the same time, here derivative of $H1$ with respect to $X1$ also has increased. So, is it not the penalty has increased? So, when penalty has increased, VAE will definitely stop that. So, this is the loss function which will definitely restrict it. So, are you understanding the analogy here? So, what exactly is

[24:14] this explanation overall is? How is the model understands? The model is understanding just with this one mathematical formula. And how it is achieved? What is this particular important analogy is saying? Whenever there are most important features, hidden representation has to react. How are you ensuring that the model is that the model ensures that the hidden representation reacts? Suppose if the hidden representation neurons doesn't react, then the features are not fully captured. If the features are not fully captured, reconstruction will be very poor. Then reconstruction loss increases. When reconstruction loss increases, it tells the model that react to those features.

[25:01] Right? Other side, look at this particular analogy where this variations has occurred because of noise or random disturbances. Then these changes are not at all important. Suppose if the model captures these, if the model captures the derivative has increased, when the derivative has increased Jacobian matrix values have increased, then the squared values have increased, then the overall penalty has increased. If the penalty has increased, we will always ensure that the penalty is always less.

[25:37] Then second confirmation is of course penalty has increased, but the second at the same time when those noise is being captured, with that when you reconstruct and when reconstructed one when you compare with the original input, original input is without noise. So there also you are getting at the reconstruction loss more and the penalty is also more. With this, the model is understanding that you should not capture those features. So this is one of the most important regularized variant of autoencoders which is a contractive, which is mainly ensuring that reconstruct the input well, but keep the hidden representation stable. So with this, we have come to an end of this particular session. So far in the last

[26:31] including this session and the last two sessions, we have seen the regularization techniques in autoencoders starting with denoising, sparse and then contractive. So in the upcoming session, we will see one numerical example and this example will make us to understand the complete auto encoder working mechanism clear and we'll also see what are the limitations of auto encoders. Thank you.